

NexDerm
AI Skin Lesion Classification
Group 11
Reflection Document
Github Repo Link : <https://github.com/NexDerm>

Divye Goswami	goswad3@mcmaster.ca	400362533
Tanvir Singh	singht48@mcmaster.ca	400378756
Hetanshu Pandya	pandyh6@mcmaster.ca	400371731
Ishaan Jamwal	jamwali@mcmaster.ca	400389636
Harkunwar Singh	singh42@mcmaster.ca	400380573

Executive Summary

NexDerm is an AI-powered dermatology support system that combines image-based skin screening with patient reports and doctor review workflows. It integrates mobile image capture, machine learning inference, input validation, OTP-based authentication, asynchronous report generation, cloud storage, and role-based clinical review, thus creating a complete end-to-end pipeline for practical healthcare support.

1. Why the Project Is Highly Complex (Main Technical Challenges Solved)

NexDerm is highly complex because it solves an end-to-end product problem, not a single-model research task. The major challenges addressed were:

- **Reliable multi-model AI inference:** A weighted ensemble was implemented using multiple (5 Each) DenseNet and ResNet checkpoints, with consistency checks to prevent class-index mismatch across models.
- **Input safety before diagnosis:** A CLIP-based skin/non-skin filter was introduced before disease prediction to reject irrelevant images and reduce unsafe outputs.
- **Fast UX with heavy backend processing:** The system separates immediate prediction response from heavier tasks like AI text generation, PDF rendering, S3 upload, and database updates via background processing.
- **Security and identity orchestration:** Authentication and OTP-based verification were integrated with Supabase-backed identity flows, while preserving protected and guest-access routes correctly.
- **Multi-role workflow management:** The same application supports guest users, authenticated patients, and doctors, including a doctor dashboard and case-review process.

2. Achievements (Brief) + AI Topline Metric

Key achievements

1. Delivered complete AI-assisted screening flow from upload to prediction.
2. Built automatic report generation pipeline with downloadable PDF output.
3. Implemented patient history timeline with status tracking and artifact retrieval.
4. Implemented doctor-in-the-loop review: pending queue, review notes, and final diagnosis.
5. Added OTP verification for stronger account security.
6. Added in-app dermatologist discovery map experience.

Topline AI metric

1. Runtime ensemble scale: 10-model inference setup (5 DenseNet + 5 ResNet).
2. Safety gate metric: CLIP skin filter threshold configured at 0.7.
3. Uncertainty metric: low-confidence warning triggered below 60% confidence.
4. We first evaluated the ensemble on the original test folder from our dataset and obtained results close to 99%. After reviewing that test set, we found many images that appeared very similar to the training data, which likely inflated performance and made the evaluation overly optimistic. To get a more realistic estimate, we then handpicked an additional dataset of 200 images and combined it with the existing test set. On this combined

evaluation set, the ensemble achieved an accuracy of 0.8832, macro precision of 0.8526, macro recall of 0.8527, and macro F1-score of 0.8513, which better reflects real-world generalization. Images for reference : [Confusion Matrix](#), [ROC AUC](#).

3. Algorithmic Innovation

The core innovation in NexDerm is a two-stage decision architecture designed for practical robustness rather than single-pass classification.

Stage A: Semantic validity filtering

Before disease inference, the system uses a CLIP-based classifier to estimate whether an uploaded image is a valid skin-related input. If the image is likely non-skin (for example, background objects or irrelevant scenes), inference is blocked and the user receives a corrective message. This stage reduces false confidence on out-of-domain data and improves operational safety.

Stage B: Weighted heterogeneous ensemble diagnosis

For valid skin images, prediction is performed by combining outputs from two backbone families (DenseNet and ResNet), each loaded from multiple checkpoints. Instead of naive averaging, NexDerm uses weighted voting and confidence aggregation, with deterministic tie resolution. This gives better stability than relying on one model and helps reduce single-checkpoint variance.

4. What the Application Does Uniquely

NexDerm’s unique outcome is that it closes the loop between AI screening and actionable follow-up:

1. Users receive prediction, confidence, risk framing, and understandable report content, not just a class label.
2. Reports are converted into downloadable clinical-style PDFs.
3. History is longitudinal and includes review states (pending/reviewed).
4. Doctors can actively review pending cases and submit final diagnosis/notes.
5. The app links the discovery of dermatologist in the same experience.
6. This creates a practical “screening-to-review” workflow rather than an isolated AI demo.

5. P0 and P1 Requirements with Completion Percentage

Priority	Requirement	Completion	Notes
P0	Users can continue as guest and get results without logging in	100%	Implemented in the app flow; guest mode can upload and see prediction results.
P0	Users can upload a skin image through the UI	100%	Image upload is supported from the mobile UI via gallery selection and camera capture.
P0	API to submit uploaded images to the backend for processing	100%	Implemented in the prediction endpoint and called by the frontend upload helper.
P0	Backend processes and validates uploaded images	100%	Invalid images are rejected and non-skin images are filtered before disease inference.
P0	Trained deep learning model classifies into skin disease categories or healthy	90%	Ensemble disease classifier is implemented with DenseNet and ResNet checkpoints.

P0	Backend provides an API endpoint for classification requests	100%	/api/v1/predictions/ is the classification endpoint.
P0	Application displays classification result clearly and accessibly	100%	The result screen shows prediction, confidence, risk level, and condition guidance.
P0	Notify user if confidence is below threshold	100%	Low-confidence warning is implemented in the backend response and surfaced in the UI.
P0	Users interact with the app via a mobile interface	100%	The client is a React Native mobile app with camera, history, account, and inference screens.
P0	Basic error handling for invalid input and server errors	100%	Invalid image, missing file, model failure, and backend failures are handled in both backend and frontend.
P0	Documentation of backend APIs	90%	FastAPI OpenAPI/docs are enabled in the backend app, giving built-in API documentation.
P0	Unit testing for model inference API	100%	There are extensive unit and integration tests for prediction and model behavior.
P1	Users can capture images directly using device camera	100%	Camera capture is implemented in the mobile UI.
P1	Users can create an account and log in with email/password	100%	Sign-up/sign-in flows exist with Supabase auth and OTP step.
P1	API to send user registration information to backend	100%	Registration is submitted via auth flow and backend OTP/auth endpoints; the system stores user metadata and verifies it server-side.
P1	Database with functionality and API endpoints for auth and saving user history	90%	Authentication, report history, latest report, and user info endpoints are implemented; history is persisted and queryable.
P1	If a condition is detected, suggest nearby dermatologists using geolocation	90%	The app has a dermatologist discovery screen, but the backend-side requirement is only partially matched; the frontend uses its own map and places flow.
P1	Backend integrates with a location-based API such as Google Maps Places API	85%	Nearby dermatologist discovery is implemented in the mobile map layer, not as a backend service contract.
P1	Store history of uploaded images and results for authenticated users	100%	Implemented in report history and latest report retrieval; the app shows saved cases and statuses.

P1	Intuitive mobile-friendly UI for predictions, dermatologist suggestions, and past results	100%	The app has dedicated mobile screens for inference, history, and dermatologist discovery with clear UX.
P1	Backend connects to an external API for text report generation	95%	Implemented via OpenRouter in the AI report service, with fallback text if the call fails.
P1	Generate a PDF report with the results of the visual diagnosis	100%	Implemented using structured AI text plus PDF rendering.
P1	User can view diagnosis history and compare with past diagnoses	100%	History screen shows previous predictions, statuses, doctor notes, and reviewed state, which supports comparison over time.

Highlights and Lowlights

The NexDerm project's biggest accomplishment was being able to create an end-to-end mobile platform that combined the front-end, back-end, machine learning pipeline, database functions and reporting into one continuous flow. Rather than just having a simple integrated classification model as its primary function, it also became a more comprehensive health care support application that included many features to assist with health care issues like Guest Access, Account Authenticated User History, Dermatologist Lookup, PDF Report Generation, and providing feedback on predictions with a confidence rating. One of the other major technical achievements was using a CLIP-based technique to filter out invalid skin images (as opposed to non-skin input) from being processed for classification purposes. A final technical achievement for the NexDerm team was collaborating across multiple technology areas at once: Frontend, Backend API's, ML Development, DB Integration, User Accounts, & Testing.

At the same time, one of the main lowlights of the project was the difficulty of keeping the implementation, documentation, and planned requirements fully aligned as the system evolved. Several technical decisions changed throughout development, which meant that parts of the original design and requirements documents had to be revisited and corrected later. In addition, integrating many independent subsystems introduced significant debugging and coordination challenges. The team also faced the broader limitation that a model performing well on a curated dataset may still struggle on real-world mobile images, making generalization and reliability an ongoing concern. Time constraints also limited the degree to which every planned feature and non-functional requirement could be polished and fully validated.

What We Learnt

This project provided valuable experience in designing and building a complete AI-assisted mobile application rather than only an isolated machine learning model. We learned how to connect a React Native frontend, a FastAPI backend, a PostgreSQL/Supabase database, and a machine learning inference pipeline into a single cohesive system. We also gained a clearer understanding of the practical challenges involved in applying AI within a healthcare-support setting, especially with respect to uncertainty, privacy, usability, and responsible design. Beyond technical implementation, we learned the importance of maintaining consistency between system requirements, design documents, and the actual codebase. Finally, the project reinforced that testing, validation, and system integration are just as important as building the core functionality itself.